Module – 5

I/O Port Interfacing & Programming

SYLLABUS:

I/O Port Interfacing & Programming: I/O Programming in 8051 C, LCD interfacing, DAC 0808 Interfacing, ADC 0804 interfacing, Stepper motor interfacing, DC motor control & Pulse Width Modulation (PWM) using C only. **(Text book 1- 7.2, 12.1, 13.1, 13.2, 17.2, 17.3)**

Table of Contents

TEXT, REFERENCE & ADDITIONAL REFERENCE BOOKS

	BOOK TITLE/AUTHORS/PUBLICATION/LINK
T-1	"The 8051 Microcontroller and Embedded Systems - Using Assembly and C", Muhammad Ali Mazidi and Janice Gillespie Mazidi and Rollind. Mckinlay; Phi, 2006 / Pearson, 2006.
T-1	"The 8051 Microcontroller", Kenneth j. Ayala, 3rd edition, Thomson/Cengage Learning.
T-1	"Programming and Customizing The 8051 Microcontroller", Myke Predko, Tata Mc Graw-Hill Edition 1999 (reprint 2003).
R-1	"The 8051 Microcontroller Based Embedded System", Manish K Patel, McGraw Hill, 2014, ISBN: 978-93-329-0125-4.

R-2	Microcontrollers: Architecture, Programming, Interfacing and System Design", Raj Kamal, Pearson Education, 2005

5.1 I/O Programming in 8051 C

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
(signed) char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65535
(signed) int	16-bit	-32768 to +32767
sbit	1-bit	SFR bit-addressable only
bit	1-bit	RAM bit-addressable only
sfr	8-bit	RAM addresses 80 – FFH only

Example 1: Write an 8051 C program to send values 00 – FF to port P1.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char z;
for (z=0;z<=255;z++)
P1=z;
}
```

Example 2: Write an 8051 C program to send hex values for ASCII characters of 0, 1, 2, 3, 4, 5, A, B, C, and D to port P1.
Solution:

```
#include <reg51.h>
```

```
void main(void)
{
unsigned char mynum[]="012345ABCD";
unsigned char z;
for (z=0;z<=10;z++)
P1=mynum[z];
}
```

Example 3: Write an 8051 C program to toggle all the bits of P1 continuously.
Solution:

```
//Toggle P1 forever
#include <reg51.h>
void main(void)
{
for (;;)
{
p1=0x55;
p1=0xAA;
}
}
```

Example 4: Write an 8051 C program to toggle bits of P1 ports continuously with a 250 ms.
Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
while (1) //repeat forever
{
p1=0x55;
MSDelay(250);
p1=0xAA;
MSDelay(250);
}
}
void MSDelay(unsigned int itime)
{
unsigned int i,j;
for (i=0;i<itime;i++)
for (j=0;j<1275;j++);
 }
```

5.2 LCD Interfacing:

LCD is finding widespread use replacing LEDs for the following reasons:

- The declining prices of LCD
- The ability to display numbers, characters, and graphics
- Ease of programming for characters and graphics.

5.2.1 Pin Description

Pin Description:

Pin	Symbol	I/O	Description
1	V_{SS}	--	Ground
2	V_{CC}	--	+5V power supply
3	V_{EE}	--	Power supply to control contrast
4	RS	I	RS=0 to select command register, RS=1 to select data register
5	R/W	I	R/W=0 for write, R/W=1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

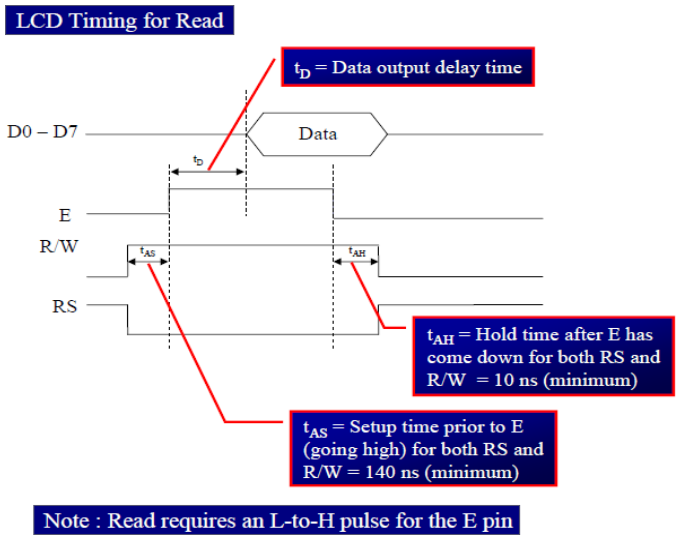
- Send displayed information or instruction command codes to the LCD
- Read the contents of the LCD's internal registers

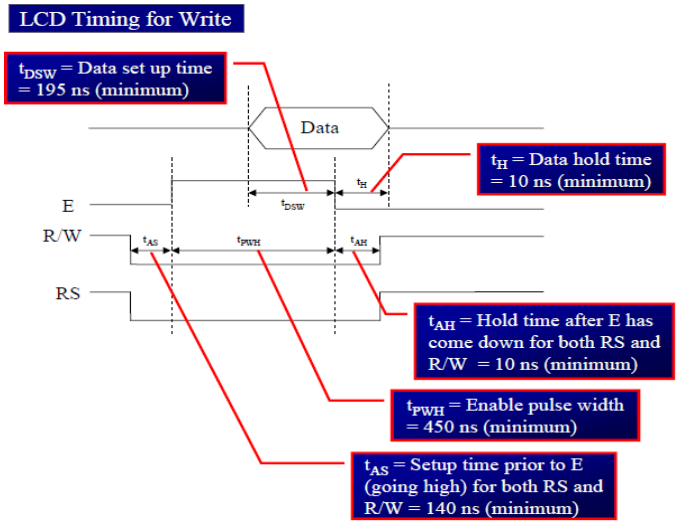
5.2.2 LCD Commands

Code (Hex)	Command to LCD Instruction Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
F	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning to 1st line
C0	Force cursor to beginning to 2nd line
38	2 lines and 5x7 matrix

5.2.3 LCD Timing Diagram:

LCD timing diagram for reading and writing is as shown in the below figures.





Sending Data/ Commands to LCDs with Time Delay:

To send any of the commands to the LCD, make pin RS=0. For data, make RS=1. Then send a high-to-low pulse to the E pin to enable the internal latch of the LCD.

Example 11: Write an ALP to initialize the LCD and display message "YES". Say the command to be given is :38H (2 lines ,5x7 matrix), 0EH (LCD on, cursor on), 01H (clear LCD), 06H (shift cursor right), 86H (cursor: line 1, pos. 6)

Program:

```
;calls a time delay before sending next data/command ;P1.0-P1.7 are connected to
LCD data pins D0-D7 ;P2.0 is connected to RS pin of LCD ;P2.1 is connected to
R/W pin of LCD ;P2.2 is connected to E pin of LCD
ORG 0H
        MOV A,#38H          ;INIT. LCD 2 LINES, 5X7 MATRIX
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#0EH          ;display on, cursor on
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY          ;give LCD some time
        MOV A,#01           ;clear LCD
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#06H          ;shift cursor right
        ACALL COMNWRT        ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#86H          ;cursor at line 1, pos. 6
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
    MOV A,#'Y'              ;display letter Y
    ACALL DATAWRT           ;call display subroutine
    ACALL DELAY             ;give LCD some time
    MOV A,#'E'              ;display letter E
    ACALL DATAWRT            ;call display subroutine
    ACALL DELAY             ;give LCD some time
    MOV A,#'S'              ;display letter S
    ACALL DATAWRT           ;call display subroutine
AGAIN: SJMP AGAIN       ;stay here

COMNWRT:                ;send command to LCD
    MOV P1,A                ;copy reg A to port 1
    CLR P2.0                ;RS=0 for command
    CLR P2.1                ;R/W=0 for write
    SETB P2.2                ;E=1 for high pulse
    ACALL DELAY             ;give LCD some time
    CLR P2.2                ;E=0 for H-to-L pulse
RET
```

```
DATAWRT:            ;write data to LCD
        MOV P1,A            ;copy reg A to port 1
        SETB P2.0           ;RS=1 for data
        CLR P2.1            ;R/W=0 for write
        SETB P2.2           ;E=1 for high pulse
        ACALL DELAY         ;give LCD some time
        CLR P2.2            ;E=0 for H-to-L pulse
RET

DELAY:
        MOV R3,#50          ;50 or higher for fast CPUs
HERE2: MOV R4,#255          ;R4 = 255
HERE: DJNZ R4,HERE          ;stay until R4 becomes 0
        DJNZ R3,HERE2
RET
END
```

5.3 Digital-to-Analog (DAC) converter:

The DAC is a device widely used to convert digital pulses to analog signals. The two methods of creating a DAC is binary weighted and R/2R ladder. The Binary Weighted DAC, which contains one resistor or current source for each bit of the DAC connected to a summing point. These precise voltages or currents sum to the correct output value. This is one of the fastest conversion methods but suffers from poor accuracy because of the high precision required for each individual voltage or current. Such high- precision resistors andcurrent-sources are expensive, so this type of converter is usually limited to 8-bit resolution or less.

The R-2R ladder DAC, which is a binary weighted DAC that uses a repeating cascaded structure of resistor values R and 2R. This improves the precision due to the relative ease of producing equal valued matched resistors (or current sources). However, wide converters perform slowly due to increasingly large RC-constants for each added R-2R link.

The first criterion for judging a DAC is its resolution, which is a function of the number of binary inputs. The common ones are 8, 10, and 12 bits. The number of data bit inputs decides the resolution of the DAC since the number of analog output levels is equal to 2^n, where n is the number of data bit inputs. Therefore, an 8-input DAC such as the DAC0808 provides 256 discrete voltage (or current) levels of output. Similarly, the 12-bit DAC provides 4096 discrete voltage levels.

DAC0808: The digital inputs are converted to current Iout, and by connecting a resistor to the Iout pin, we can convert the result to voltage. The total current Iout is a function of the binary numbers at the D0-D7 inputs of the DAC0808 and the reference current Iref , and is as follows:

$$I_{out} = I_{ref}\left(\frac{D7}{2} + \frac{D6}{4} + \frac{D5}{8} + \frac{D4}{16} + \frac{D3}{32} + \frac{D2}{64} + \frac{D1}{128} + \frac{D0}{256}\right)$$

Usually reference current is 2mA. Ideally we connect the output pin to a resistor, convert this current to voltage, and monitor the output on the scope. But this can cause inaccuracy; hence an opamp is used to convert the output current to voltage. The 8051 connection to DAC0808 is as shown in the figure below.

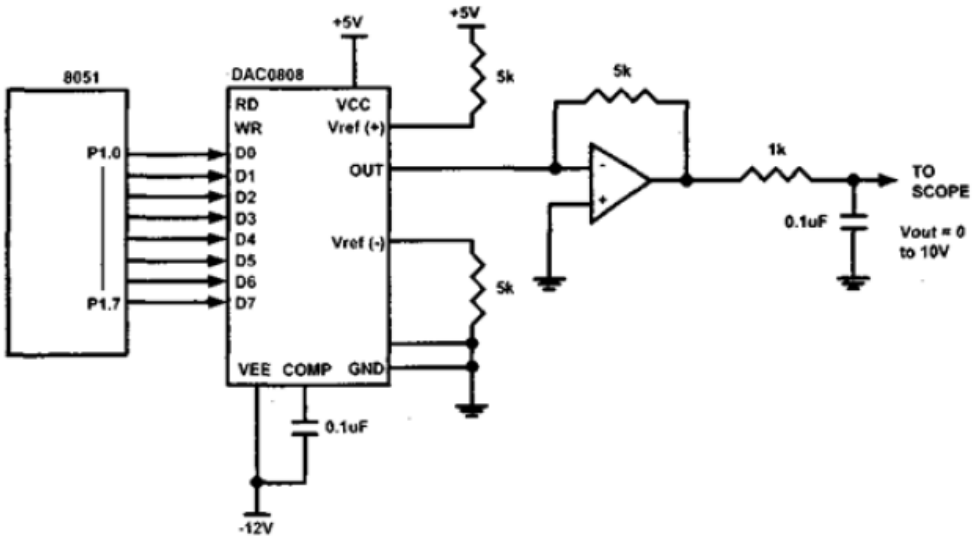


Figure: 8051 connection to DAC0808

5.3.1 Sawtooth waveform:

```
        MOV A, #00H
        MOV P1, A
BACK:   INC A
        SJMP BACK
```

```
        MOV R0, #64H
RPT:    MOV A, #00H
BACK:   MOV P1, A
        INC A
        CJNE A, R0, BACK
        SJMP RPT
```

5.3.2 Triangular waveform:

```
        MOV A, #00H
INCR:   MOV P1, A
        INC A
        CJNE A, #255, INCR
DECR:   MOV P1, A
        DEC A
        CJNE A, #00, DECR
        SJMP INCR
        END
```

5.3.3 Staircase waveform:

```
        MOV A, #00H
        MOV P1, A
        ACALL DELAY
RPT: ADD A, #51
        MOV P1, A
        ACALL DELAY
        CJNE A, #255, RPT
DELAY:
        MOV R0, #64H
RPT: MOV A, #00H
BACK: MOV P1, A
        INC A
        CJNE A, R0, BACK
        SJMP RPT
```

5.3.4 Sine Waveform:

Write an ALP to generate a sine waveform. Vout= 5V(1+sinθ)

Solution:

Calculate the decimal values for every 10 degree of the sine wave. These values can be maintained in a table and simply the values can be sent to port P1. The sine wave can be observed on the CRO.

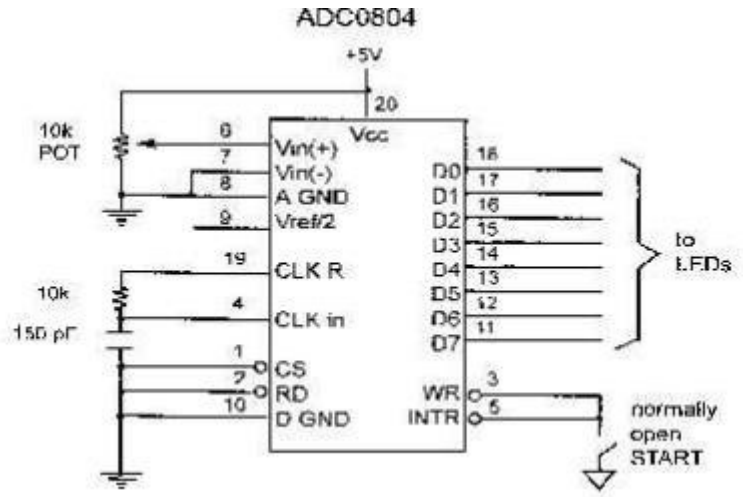
```
        ORG 0000H
AGAIN:  MOV DPTR, #SINETABLE
        MOV R3, #COUNT
UP:     CLR A
        MOVC A, @A+DPTR
        MOV P1, A
        INC DPTR
        DJNZ R3, UP
        SJMP AGAIN
        ORG 0300H
SINETABLE DB 128, 192, 238, 255, 238, 192, 128, 64, 17, 0, 17, 64, 128
        END
```

5.4 Analog-to-digital converter (ADC) interfacing:

ADCs (analog-to-digital converters) are among the most widely used devices for data acquisition. A physical quantity, like temperature, pressure, humidity, and velocity, etc., is converted to electrical (voltage, current) signals using a device called a transducer, or sensor. We need an analog-to-digital converter to translate the analog signals to digital numbers, so microcontroller can read them.

ADC804 chip: ADC804 IC is an analog-to-digital converter. It works with +5 volts and has a resolution of 8 bits. Conversion time is another major factor in judging an ADC. Conversion time is defined as the time it takes the ADC to convert the analog input to a digital (binary) number. In ADC804 conversion time varies depending on the clocking signals applied to CLK R and CLK IN pins, but it cannot be faster than 110µs.

Pin Description of ADC804:



CLK IN and CLK R: CLK IN is an input pin connected to an external clock source. To use the internal clock generator (also called self-clocking), CLK IN and CLK R pins are connected to a capacitor and a

resistor and the clock frequency is determined by:
Typical values are R = 10K ohms and C =150pF. We get f = 606 kHz and the conversion time is 110μs.
Vref/2 : It is used for the reference voltage. If this pin is open (not connected), the analog input voltage is in the range of 0 to 5 volts (the same as the Vcc pin). If the analog input range needs to be 0 to 4volts, Vref/2 is connected to 2 volts. Step size is the smallest change can be discerned by an ADC

Vref/2 Relation to Vin Range

$V_{ref}/2$(v)	V_{in}(V)	Step Size (mV)
Not connected*	0 to 5	5/256=19.53
2.0	0 to 4	4/255=15.62
1.5	0 to 3	3/256=11.71
1.28	0 to 2.56	2.56/256=10
1.0	0 to 2	2/256=7.81
0.5	0 to 1	1/256=3.90

D0-D7: The digital data output pins. These are tri-state buffered. The converted data is accessed only when CS =0 and RD is forced low. To calculate the output voltage, use the following formula

$$D_{out} = \frac{V_{in}}{step\ size}$$

Dout = digital data output (in decimal),
Vin = analog voltage, and
step size (resolution) is the smallest change

Analog ground and digital ground: Analog ground is connected to the ground of the analog Vin and digital ground is connected to the ground of the Vcc pin. To isolate the analog Vin signal from transient voltages caused by digital switching of the output D0 – D7. This contributes to the accuracy of the digital data output.

Vin(+) & Vin(-): Differential analog inputs where Vin = Vin (+) – Vin (-). Vin (-) is connected toground and Vin (+) is used as the analog input to be converted.

RD: Is "output enable" a high-to-low RD pulse is used to get the 8-bit converted data out of ADC804.

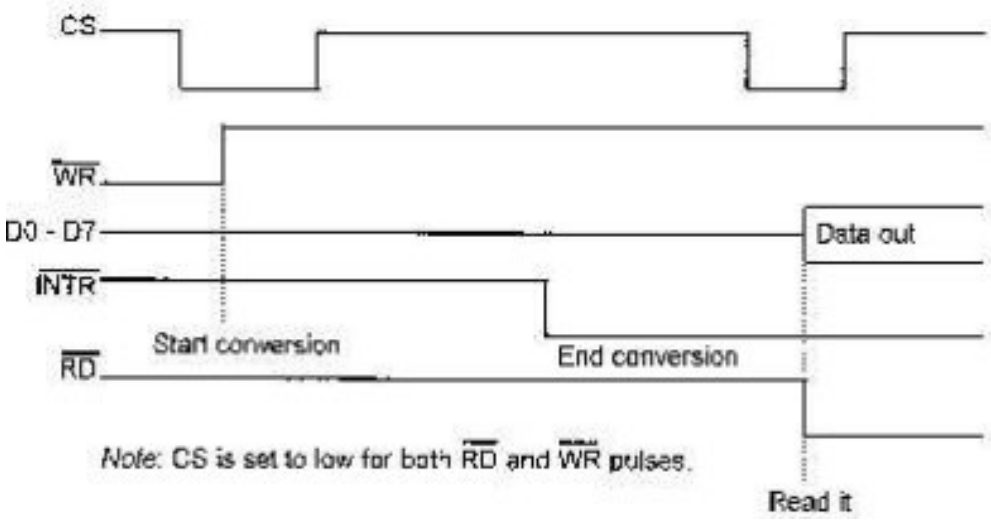
INTR: It is "end of conversion" When the conversion is finished, it goes low to signal the CPU thatthe converted data is ready to be picked up.

WR: It is "start conversion" When WR makes a low-to-high transition, ADC804 starts converting the analog input value of Vin to an 8- bit digital number.

CS: It is an active low input used to activate ADC804.

The following steps must be followed for data conversion by the ADC804 chip:
1. Make CS= 0 and send a L-to-H pulse to pin WR to start conversion.
2. Monitor the INTR pin, if high keep polling but if low, conversion is complete, go to next step.
3. Make CS= 0 and send a H-to-L pulse to pin RD to get the data out

The following figure shows the read and write timing for ADC804.



Write a program to monitor the INTR pin and bring an analog input into register A. Then call a hex-to ACSII conversion and data display subroutines. Do this continuously.
;p2.6=WR (start conversion needs to L-to-H pulse)
;p2.7 When low, end-of-conversion)
;p2.5=RD (a H-to-L will read the data from ADC chip)
;p1.0 – P1.7= D0 - D7 of the ADC804

```
        MOV P1, #0FFH       ;make P1 = input
BACK:   CLR P2.6            ;WR = 0
        SETB P2.6           ;WR = 1 L-to-H to start conversion
HERE:   JB P2.7,HERE        ;wait for end of conversion
        CLR P2.5            ;conversion finished, enable RD
        MOV A,P1            ;read the data
        SETB p2.5           ;make RD=1 for next round
        SJMP BACK
```

5.5 Stepper Motor Interfacing:

Stepper motor is a widely used device that translates electrical pulses into mechanical movement. Stepper motor is used in applications such as; disk drives, dot matrix printer, robotics etc,. The construction of the motor is as shown in figure below.

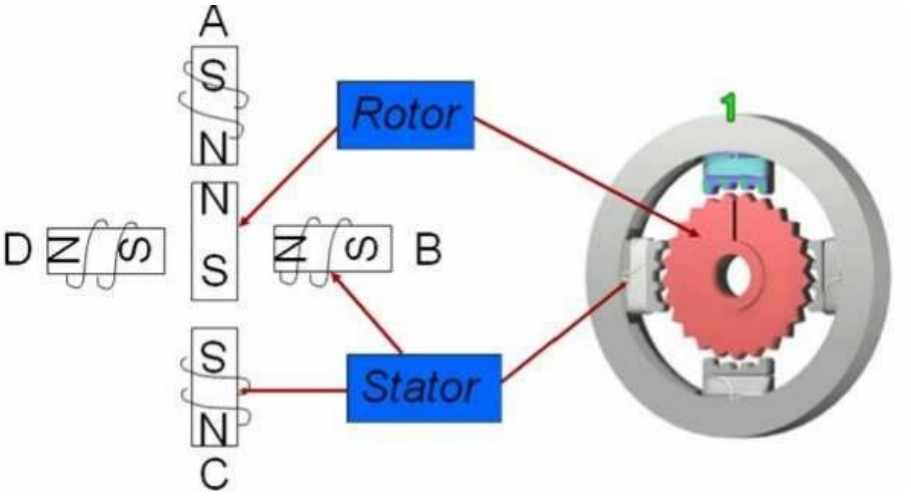


Figure: Structure of stepper motor

It has a permanent magnet rotor called the shaft which is surrounded by a stator. Commonly used stepper motors have four stator windings that are paired with a center – tapped common. Such motors are called as four-phase or unipolar stepper motor. The stator is a magnet over which the electric coil is wound. One end of the coils are connected commonly either to ground or +5V. The other end is provided with a fixed sequence such that the motor rotates in a particular direction. Stepper motor shaft moves in a fixed repeatable increment, which allows one to move it to a precise position. Direction of the rotation is dictated by the stator poles. Stator poles are determined by the current sent through the wire coils.

Step angle: Step angle is defined as the minimum degree of rotation with a single step.
No of steps per revolution = 360° / step angle
Steps per second = (rpm x steps per revolution) / 60

Example: step angle = 2°
No of steps per revolution = 180

Switching Sequence of Motor: As discussed earlier the coils need to be energized for the rotation.This can be done by sending a bits sequence to one end of the coil while the other end is commonlyconnected. The bit sequence sent can make either one phase ON or two phase ON for a full step sequence or it can be a combination of one and two phase ON for half step sequence. Both are tabulated below.

Full Step: Two Phase ON

Clockwise

Step #	A	B	C	D
1	1	0	0	1
2	1	1	0	0
3	0	1	1	0
4	0	0	1	1

Counter-clockwise

One Phase ON

Clockwise

Step #	A	B	C	D
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

Counter-clockwise

Half Step (8 – sequence): The sequence is tabulated as below:

Clk

Step #	A	B	C	D
1	1	0	0	1
2	1	0	0	0
3	1	1	0	0
4	0	1	0	0
5	0	1	1	0
6	0	0	1	0
7	0	0	1	1
8	0	0	0	1

Anti-Clk

8051 Connection to Stepper Motor: (explanation of the diagram can be done)

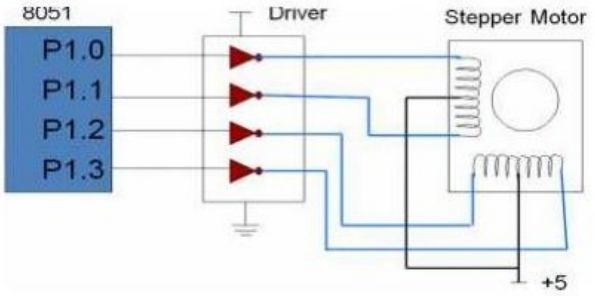


Figure: 8051 interface to stepper motor

The following example 1 to example 3 shown below will elaborate on the discussion done above:

Example 1: Write an ALP to rotate the stepper motor clockwise / anticlockwise continuously with fullstep sequence.

Program:

```
        MOV  A, #66H        ; Load accumulator with initial pattern 66H (0110 0110)
BACK:   MOV P1, A           ;Send accumulator contents to Port 1
                            ; P1.0 – P1.3 are connected to motor driver

        RRA                 ; Rotate accumulator right, Generates next stepping sequence
                            ;automatically
        ACALL DELAY         ;
        SJMP  BACK
DELAY:  MOV  R1, #100
UP1:    MOV  R2, #50
UP:     DJNZ R2, UP
        DJNZ  R1, UP1
        RET
```

Note: motor to rotate in anticlockwise use instruction RL A instead of RR A

Example 2: A switch is connected to pin P2.7. Write an ALP to monitor the status of the SW. If SW = 0, motor moves clockwise and if SW = 1, motor moves anticlockwise.

Program:

```
        ORG 0000H
        SETB P2.7
        MOV A, #66H
        MOV P1,A
TURN: JNB P2.7, CW
        RL A
        ACALL DELAY
        MOV P1,A
        SJMP TURN
CW:   RR A
        ACALL DELAY
        MOV P1,A
        SJMP TURN
DELAY: as previous example
```

Example 3: Write an ALP to rotate a motor 90° clockwise. Step angle of motor is 2°.

Solution:
Step angle = 2°
Steps per revolution = 180
No of rotor teeth = 45
For 90° rotation the no of steps is 45

Program:

```
        ORG 0000H
        MOV A, #66H
        MOV R0, #45
BACK: RR A
        MOV P1, A
        ACALL DELAY
        DJNZ R0, BACK
        END
```

5.6 DC Motor Interfacing:

DC motor is a device that translates electrical pulses into mechanical movement

➢ The DC motor has + and – leads

➢ Connecting them to a DC voltage source moves the motor in one direction and by reversing the polarity, the DC motor will move in opposite direction.

Unidirectional Control:

➢ The following figure shows the DC motor rotation for clockwise (CW) and counterclockwise (CCW) rotations.

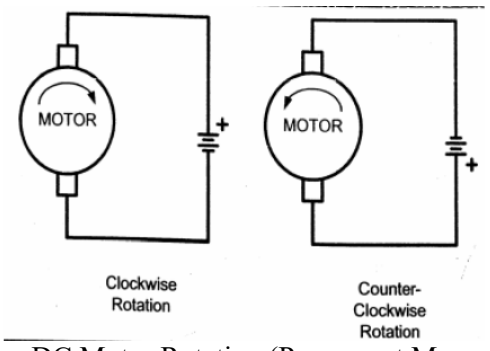
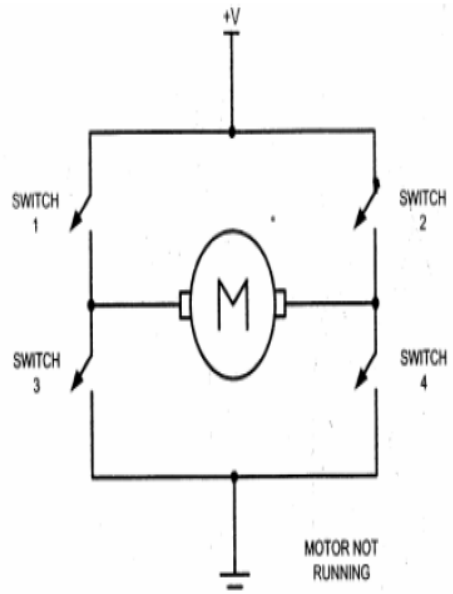


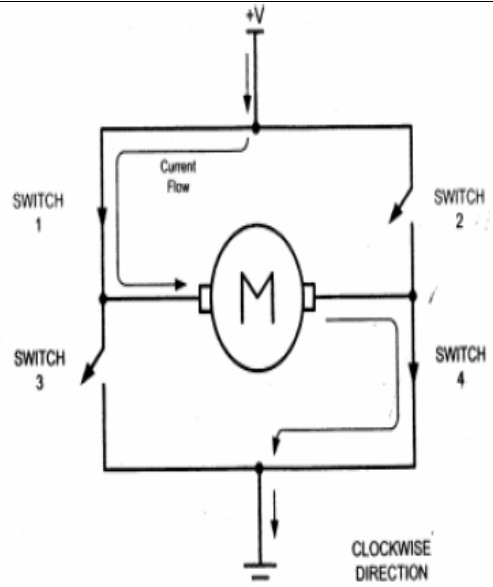
Figure: DC Motor Rotation (Permanent Magnet Field)

Bidirectional Control:

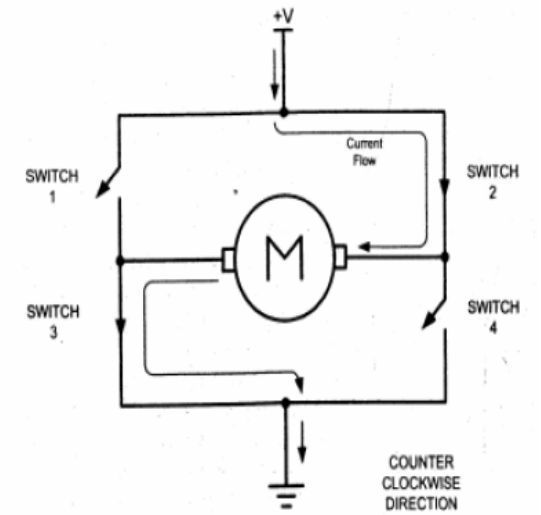
- With the help of relays or some specially designed chips we can change the direction of the DC motor rotation. Below Figure shows the basic concepts of H-Bridge control of DC motors.



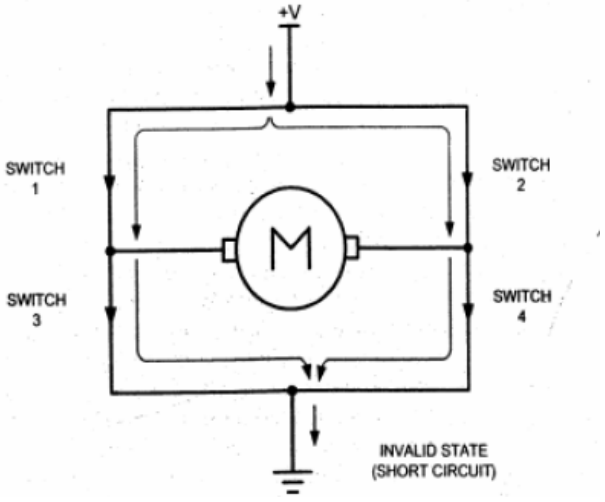
H-Bridge Motor Configuration
All the switches are open, which does not allow the motor to turn



H-Bridge Motor Clockwise Configuration
When switches 1 and 4 are closed, current is allowed to pass through the motor in one direction



H-Bridge Motor Counterclockwise Configuration
When switches 2 and 3 are closed, current is allowed to pass through the motor in the opposite direction



H-Bridge in an invalid configuration.
Current flows directly to ground, creating a short circuit

Example: A switch is connected to pin P2.7. Write a C program to monitor the status of SW and perform the following:

a) If SW=0, the DC motor moves clockwise.
b) If SW=1, the DC motor moves counterclockwise.

Solution:

```
# include <reg51.h>
sbit SW = P2^7;
sbit ENABLE = P1^0;
sbit MTR_1 = P1^1;
sbit MTR_2 = P1^2;
void main(0
{
SW = 1;
ENABLE = 0;
MTR_1 = 0;
MTR_2 = 0;
while (1)
(
ENABLE = 1;
If (SW == 1)
{
MTR_1 = 1;
MTR_2 = 0;
}
else
{
MTR_1 = 0;
MTR_2 = 1;
}
}
}
```

Example: Write a C program to monitor the status of SW and perform the following:

a) If SW=0, the DC motor moves with 50% duty cycle pulse
b) If SW=1, the DC motor moves with 25% duty cycle pulse

Solution:

```
# include <reg51.h>
sbit SW = P2^7;
sbit MTR = P1^0;

void MSDelay(unsigned int value);
void main( )
{
```

```
SW = 1;
MTR = 0;
while (1)
{
If (SW == 1)
{
MTR = 1;
MSDelay(25);
MTR = 0;
MSDelay(75);
}
else
{
MTR = 1;
MSDelay(50);
MTR = 0;
MSDelay(50);
}
}
}
void MSDelay(unsigned int value);
{
unsigned char x,y;
for (x=0; x<1275; x++)
    for (y=0; y<value; y++);
}
```

5.7 Question Bank

1. Explain DAC interface with diagram and also write a program to generate staircase waveform
2. Explain DAC interface with diagram and also write a program to generate triangular waveform
3. Write a program to display "HELLO WORLD" by interfacing LCD display to 8051 microcontroller
4. Explain stepper motor interface with diagram and also write a program to monitor the status of switch and rotate clockwise if status of switch is zero and anticlockwise if status of switch is one.
5. Explain the interfacing of DC motor using C programming.
6. With neat diagram write an assembly language program to interface DAC to 8051 microcontroller.
7. With neat diagram write an assembly language program to interface LCD to 8051 microcontroller.
8. With neat diagram write an assembly language program to interface Stepper motor to 8051 microcontroller
9. With neat diagram write an assembly language program to interface ADC0804 to 8051 microcontroller.
10. A door sensor is connected to the P1.1 pin and a buzzer is connected to P1.7. Write 8051 C program to monitor the door sensor and when it opens, sound the buzzer. The buzzer can be sound by sending a square wave of a few hundred Hz.